

(1, 5, 10, 15, 20)

M T W T F S S
Page No.:
Date:
YOUVA

[A1] :- Assignment No: 02

- ✓ 1] Explain Depth limited Search Algorithm with suitable examples.
- ✓ 5] What are the characteristics of heuristic search? Explain properties of a Heuristic Search Algorithm.
- ✓ 10] Differentiate Hill Climbing vs A* Algorithm.
- ✓ 15] Explain forward chaining Algorithm with the help example.
- ✓ 20] Formulate state space search problem for vacuum world cleaner.

UT2 Imps = (12) (5) (8) (10) (15) (20)

M T W T F S S
Page No.:
Date:
YOUVA

[A1] :- UT2 Question Bank

- ✓ 2] Explain Uniform cost Search Algorithm with suitable examples.
- ✓ 3] Explain Deepening first Search Algorithm with suitable examples.
- 4] What is Heuristics function? Explain 8 puzzle problem.
- 6] Explain A* Algorithm with suitable examples.
- 7] Explain Greedy Best First Search Algorithm with suitable eg.
- ✓ 8] Explain Hill Climbing and its Drawback in details.
- 9] Explain an Advantages and Applications of Hill Climbing Algorithm.
- 11] Write a short note on: Game Playing.
- 12] Write a short note on: Genetic Algorithm.
- 13] Explain Wumpus world environment given its PEAS description.
- 14] Explain the limitations of Propositional logic with example.
- 16] Explain Backward Chaining Algorithm with the help example.
- 17] Attempt: Conditional Probability.
- 18] Write short note: Bayesian Network with example.
- 19] Write short note: Bayes Theorem.

15/11/26

* Answer for UT 2 QB:—

10QJ Differentiate: Hill Climbing vs A* Algorithm.

Parameters	Hill Climbing	A* Algorithm
Type of Search	Local Search Algo.	Informed Search Algo
Approach	Greedy Chooses best + immediate neighbor	Uses both actual cost + heuristic.
Evaluation (F^n)	Uses only heuristic $F^n h(n)$	Uses $F(n) = g(n) + h(n)$
Search Nature	Does not explore entire Search space.	Explores multiple path systematically.
Optimality	Not guarantee to find Optimal sol ⁿ	Guarantee Optimal
Completeness	Not complete	Complete.
Backtracking	No	Allow
Memory usage	Little Memory req ^d	Large Memory req ^d
Speed	Faster	Slower than hill climbing.
Sol ⁿ Quality	May give Sub-optimal sol ⁿ	Gives optimal sol ⁿ
Eg. use	Optimization problems	Pathfinding Shortest Path Prblm.

1QJ Explain "Depth Limited Search Algorithm" with suitable examples.

- Depth Limited Search Algorithm is Search Algorithm where we explore the Search tree upto fixed depth limit (L).
- It prevents DFS from going infinitely deep (which is a major problem in infinite trees)

Need DLS

In normal DFS,

- It may go into infinite depth.
- It may never find solⁿ, even if it exists.

DFS solve this by:

- Adding a maximum depth limit.
- Stopping Search beyond that level.

Basic working

- Starting from root node
- Explore nodes depth-wise (like DFS).
- Stop when

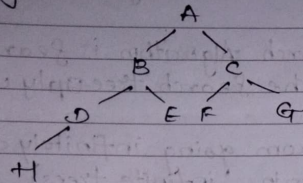
Goal is found ✓

Depth limit is reach x

• Algorithm:—

1. Start at root node.
2. If node is goal → return Success.
3. If depth == limit → return cutoff.
4. Else: • Expand node.
• Apply DLS to each child.
5. If any child return success → return success.
6. Else → return failure.

for e.g.,



Case 1 : Depth Limit = 2
levels

- Level 0 $\rightarrow$ A
- Level 1 $\rightarrow$ B, C
- Level 2 $\rightarrow$ D, E, F, G

$\therefore$ Node H is at lvl 3, so it will not be explored.

Traversal (DFS style)

A $\rightarrow$ B $\rightarrow$ D $\rightarrow$ E $\rightarrow$ C $\rightarrow$ F $\rightarrow$ G

Case 2 : Depth limit = 3

Now H is within limit

Traversal

A $\rightarrow$ B $\rightarrow$ D $\rightarrow$ H $\rightarrow$ E $\rightarrow$ C $\rightarrow$ F $\rightarrow$ G

$\therefore$ Goal can be found if it is H.

Adv.

1. Prevents Infinite loops
2. Saves Memory (like DFS)
3. Useful when soln depth is known.

Disadv.

1. If limit is too small $\rightarrow$ may miss soln.
2. If limit too large $\rightarrow$ behave like DFS.
3. Not Optimal (may not give shortest path)

2Q] Explain Uniform Cost Search Algorithm with suitable examples.

$\rightarrow$ Uniform Cost Search (UCS) is an Uninformed Search algorithm used in Artificial Intelligence to find the minimum cost path from a Start node to a Goal node.

- UCS always select least expensive path. it guarantees finding the optimal (least cost) soln provided all step costs are non-negative.

Basic idea

- Instead of expanding nodes level by level (like BFS), UCS expands nodes based on path cost.
- It uses a priority queue where, priority = lowest path cost
- The node with smallest cost is expanded first

Working principle :

1. Start with Initial node (cost = 0).
2. Insert it into a priority queue.
3. Repeat the following steps :
 - Remove the node with the lowest cost.
 - If it is the goal node $\rightarrow$ return soln.
 - Otherwise, expand the node.
 - Add all child nodes to the queue with updated costs.

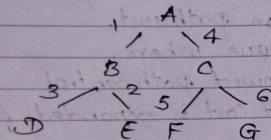
4. Continue until the goal node is reached.

Algorithm

Step-by-step procedure

1. Create a priority queue (PQ).
2. Insert start node, with cost = 0
3. While PQ is not empty
 - Remove node with minimum cost.
 - If node is goal $\rightarrow$ return path.
 - Else expand node.
 - Add children with updated path cost.

Eg.,



Step-by-step Execution

Step 1: Start at node A.

Cost = 0

Queue: (A, 0)

Step 2: Expand A

Add

B $\rightarrow$ cost = 1

C $\rightarrow$ cost = 4

Queue: (B, 1), (C, 4)

Step 3: Expand B (lowest cost)

Add

D $\rightarrow$ 1 + 3 = 4

E $\rightarrow$ 1 + 2 = 3

Queue: (E, 3), (C, 4), (D, 4)

Step 4: Expand E (No children)

Queue (C, 4), (D, 4)

Step 5: Expand C

Add

F $\rightarrow$ 4 + 5 = 9

G $\rightarrow$ 4 + 6 = 10

Queue: (D, 4), (F, 9), (G, 10)

Step 6: Expand D

Goal reached

Find result

Optimal path A $\rightarrow$ B $\rightarrow$ D

Total cost: 4

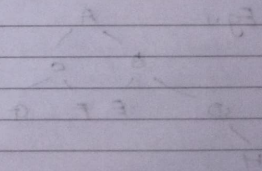
UCS always select the minimum cost path, not the shortest in no. of steps.

Adv.

1. Optimal soln
2. Suitable for weighted graph.
3. Widely used in real-world appln.

Disadv.

1. req. large memory
2. Slower when path cost large
3. Explores unnecessary paths before reaching goal.



3 Q) Explain Deepening first Search Algorithm with suitable example.

→ Iterative Deepening first Search Algo. is search technique in which Depth limited Search (DLS) is applied repeatedly with increasing depth limit until the goal node is found.

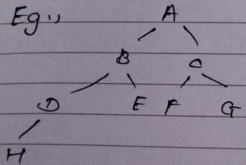
• Working Principle

1. Set depth limit $L=0$
2. Apply Depth limited Search (DLS)
3. If goal found $\rightarrow$ return soln
4. Else Increase $L=L+1$
5. Repeat until goal is found.

• Algo Steps

1. Initialize depth = 0
2. Repeat : perform DLS with given details.
 - a. If result is success $\rightarrow$ return soln
 - Else increase depth.
3. Continue until goal is reached.

Eg.,



Goal node = H

Iteration 1: Depth Limit = 0

Visited:

A (Goal not found)

Iteration 2: Depth limit = 1

Visited:

A $\rightarrow$ B $\rightarrow$ C (Goal not found)

Iteration 3: Depth limit = 2

Visited:

A $\rightarrow$ B $\rightarrow$ D $\rightarrow$ E $\rightarrow$ C $\rightarrow$ F $\rightarrow$ G (Goal not found)

Iteration 4: Depth limit = 3

Visited:

A $\rightarrow$ B $\rightarrow$ D $\rightarrow$ H (Goal found)

Final result

path A $\rightarrow$ B $\rightarrow$ D $\rightarrow$ H

Goal found at depth = 3

Adv

- 1) Combine benefits BFS & DFS
- 2) Use less memory than BFS.
- 3) Guarantee finding shallowest soln.

Disadv

- 1) Repeat nodes multiple times.
- 2) Slightly more time.

5. Q7 What are the Characteristics of Heuristic Search?
Explain properties of Heuristic Search Algorithms.

Heuristic Search uses a heuristic function ($h(n)$) that estimate the cost from a node to the goal.

* Characteristic of Heuristic Search: —

1) Use Domain Knowledge:

- It relies on heuristic function $h(n)$ that estimate how close a node is to the goal.
- This makes the search more intelligent and directed.

2) Goal Oriented:

- Instead of exploring blindly it prioritizes nodes that seem closer to the goal.
- Reduces unnecessary exploration.

3) Reduce Search Space:

- By making informed choices it avoids expanding irrelevant nodes.
- Much more efficient than uniformed search in large problem.

4) Not always Optimal, Optimal:

- If the heuristic is not well-designed the Algorithm miss the Optimal soln.

5) Performance Dependent Heuristic Quality:

- Good heuristic $\rightarrow$ fast & optimal
- Poor heuristic $\rightarrow$ behave like uninformed search.

6) Can be greedy or Balance:

7) Informed Search Technique:

- Unlike blind search, it has additional info.
- Makes better decision while exploring nodes.

* Properties of Heuristic Search Algorithms: —

① Admissibility:

An Heuristic is admissible if it never overestimates the true cost $h(n)$ is actual cost.
Guarantees optimal soln.

② Consistency:

Stronger condition than admissibility.
An heuristic is consistent if:
 $h(n) \leq \text{cost}(n, n') + h(n')$

③ Completeness:

Algo. find soln if it exist.
Eg.: A* is complete (under certain condⁿ)

④ Optimality:

Find the best (least cost) soln.
Achieved when heuristic is admissible.

⑤ Time Complexity:

Depends on quality of heuristic.
Better heuristic $\rightarrow$ fewer nodes explored.

⑥ Space Complexity:

Heuristic algo. stores nodes in memory.

8 Q Explain Hill Climbing and its drawbacks in details.

→ Hill Climbing is an Iterative Search Algo. that starts with an initial soln and repeatedly moves to a neighbouring state with a better value until no further improvement is possible.

- Key idea

- Start from an Initial State.
- Evaluate neighbouring State.
- Move to a best neighbour exists.
- It does not backtrack.

- Drawbacks :-

1. Local Maximum Problem :
Algo may stop at local peak, not global peak.
2. Plateau Problem :
Algo. gets stuck flat region where all neighbours have same value.
3. Ridge Problem :
Path to peak is narrow & zig-zag
Algo. cannot follow proper path.
4. No Backtracking :
Once it moves forward, it almost goes back.
5. No Complete :
May fail to find soln
6. No Optimal :
Does not guarantee Best soln.

15 Q Explain forward Chaining Algo. with the help of example.

→

- Forward Chaining Algo. is a data driven inference algo. used in AI.
- It starts with known facts & applies rules to derive new facts until the goal is reached.
- It mainly used
 - Expert System
 - Rule based systems

• Algorithm :-

1. Initialize
Add known facts to working memory.
2. Match Rules
Find rule whose condⁿ (IF part)
Match known facts.
3. Fire Rule
Apply the rule & add conclusion (then part)
4. Update
Add new facts to knowledge base
5. Repeat
Continue until goal is reached or rule applies

Eg.)

Given knowledge base
1

Facts:

A

B

Rules:

1. $A \wedge B \rightarrow C$

2. $C \rightarrow D$

3. $D \rightarrow \text{Goal}$

Step-by-Step Execution

Step 1: Initial Facts

A, B

Step 2: Apply Rule 1

$A \wedge B \rightarrow C$

Since A and B are true

Add C

Step 3: Apply Rule 2

$C \rightarrow D$

C is true

Add D

Step 4: Apply Rule 3

$D \rightarrow \text{Goal}$

D is true (Goal is achieved)

Inference Chain

$A, B \rightarrow C \rightarrow D \rightarrow \text{Goal}$

20. Q) Formulate State space search problem for Vacuum World Cleaner.

→ The vacuum cleaner world is a simple problem used in AI to demonstrate how an intelligent agent performs state space search to clean an environment.

- The environment consists of rooms that may be clean or dirty and the vacuum agent must clean all rooms efficiently.

- Problem Formulation.

1) State space

Location of Vacuum Cleaner (left or right)

Status of each room (Clean or dirty)

For 2-room environment (A and B)

Each room can be

Clean (C)

Dirty (D)

State

(A, D, D)

(A, C, D)

(A, D, C)

(A, C, C)

(B, D, D)

(B, C, D)

(B, D, C)

(B, C, C)

Description

Vacuum in A, both dirty room

|, A clean, B dirty

-|, A dirty, B clean

|, both clean room

Vacuum in B, both dirty

|, A clean, B dirty

-|, A dirty, B clean

|, both clean

2) Initial State

Any configⁿ can be starting state.

3) Action Operator

Vacuum agent can perform

left $\rightarrow$ Move to L Room

right $\rightarrow$ Move to right room

suck $\rightarrow$ Clean current room

4) Transition Model

Describe result of each action

If action = suck

(A, C, D) $\rightarrow$ Current room become clean

If action = left/right

$\rightarrow$ Vacuum move to respective room

5. Goal State

All rooms must be clean regardless of assumption.

Goal state

(A, C, C)

(B, C, C)

6. Path cost

Each cost of action = 1.

State space Representⁿ

(A, D, D)

$\downarrow$

Suck

$\downarrow$

(A, C, D) $\rightarrow$ right $\rightarrow$ (B, C, D)

$\downarrow$

Suck

$\downarrow$

(B, C, C) $\checkmark$ Goal.

PEAS

Performance : All rooms clean min steps

Environment : 2 room (A, B)

Actions : Move left, Move right

Sensors : Detect locⁿ & dirt status